Teoria dos Grafos

Bruno Pereira dos Santos

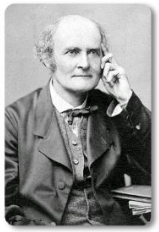
Aula baseada nos cursos de outros docentes da UFMG (Prof. Antonio Alfredo Ferreira Loureiro), USP (Prof. Paulo Feofiloff), UFBA, livros, artigos e outros.

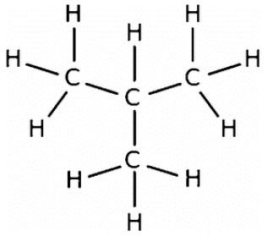
06 - Árvores

Árvores

Lembra disso?

Modelos usando grafos Estruturas de moléculas de hidrocarboneto





Arthur Cayley (1821–1895),matemático inglês. Logo após o trabalho de Kirchoff, Cayley usou "árvores matemática" para enumerar todos os isômeros para certos hidrocarbonetos.

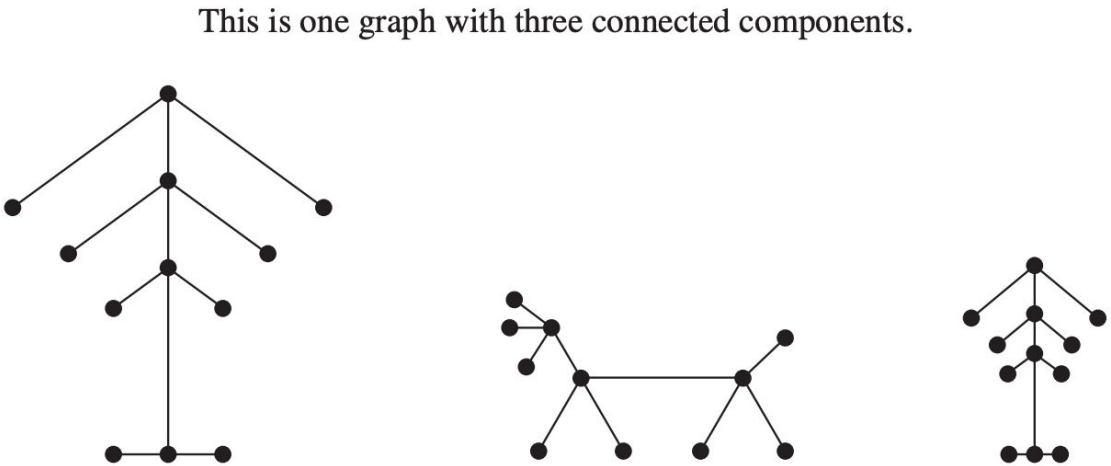
- Um grafo para hidrocarboneto $G = (V, E)$
  - **Vértices (V)** → átomos de carbono
  - **Arestas (E)** → ligações covalentes C–C
  - **Grau do vértice** → valência (máx. 4 para o carbono)
    - Grau ≤ 4
    - Grafo conexo
    - Sem ciclos (para alcanos
- Cayley estudou árvores (grafos conexos e sem ciclos) ao investigar quantas estruturas químicas distintas podem existir para uma mesma fórmula molecular.

Usos em algoritimos

- Construção eficientes de algoritmos para localização de itens
- Código de Huffman - Para construção eficiente de códigos para poupar custos de transmissão ou armazenamento
- Jogos (Cubo Mágicos)
- Sequências de Decisões
- ...

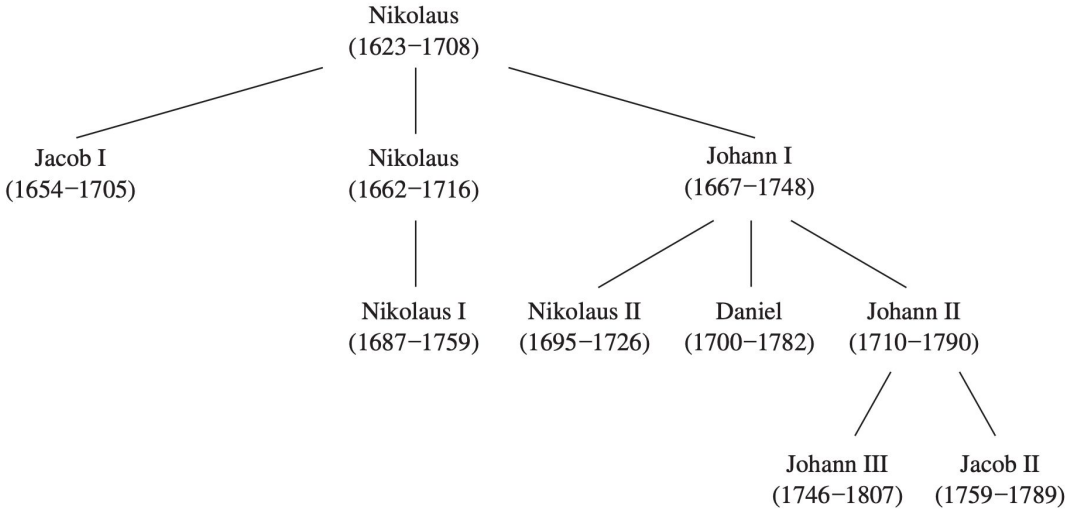
Floresta

- Definição: Uma floresta é um grafo não dirigido acíclico podendo ou não ser conexo.



Árvores

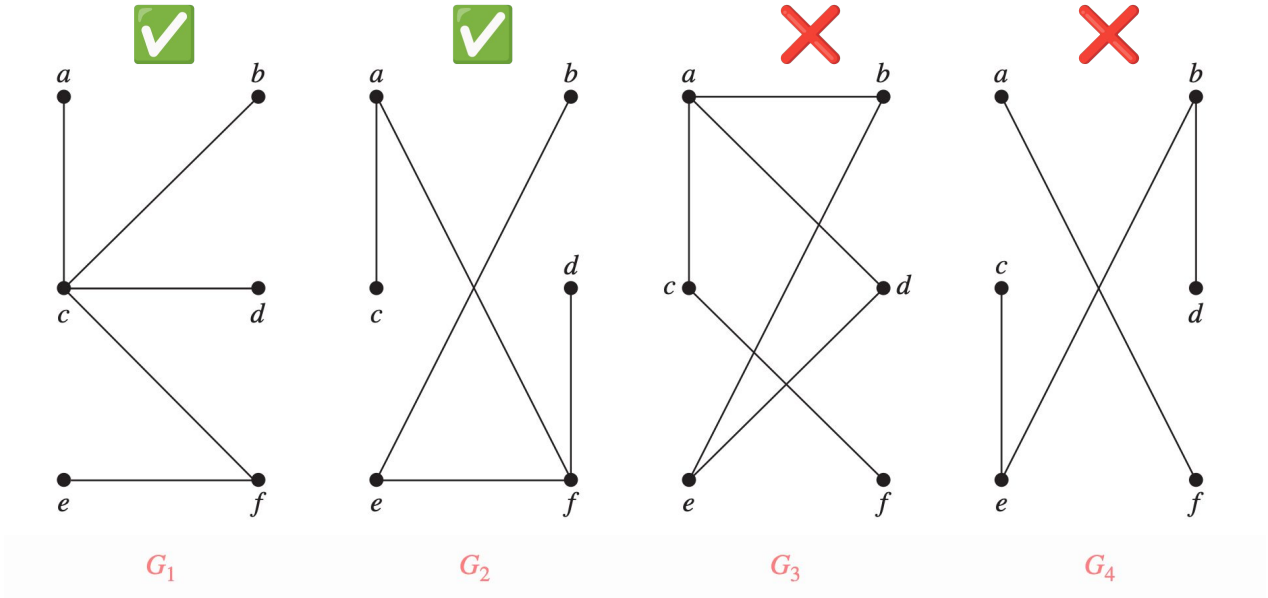
- Tipo particular de grafo
- Recebe o nome de árvore, pois lembram árvores!
- A árvore genealógica dos membros da família Bernoulli de matemáticos suíços:



Árvores (1)

- <u>Definição:</u> uma árvore é um grafo não direcionado e conexo, sem circuitos simples.

Quais são árvores?



Árvores (2)

- Teorema: Um grafo não direcionado é uma árvore se, e somente se, existe um único caminho simples entre quaisquer dois de seus vértices.

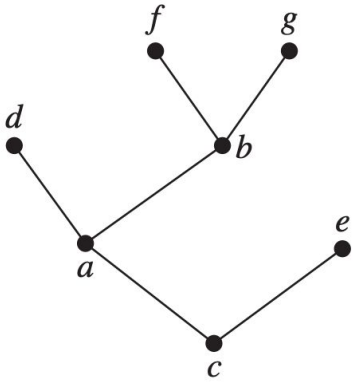
Árvores (3)

- Prova: (⇒) Se T é uma árvore
  - Uma árvore é um grafo conexo e sem circuitos simples.
  - Como é conexo, existe pelo menos um caminho simples entre quaisquer vértices x e y.
  - Suponha que existam dois caminhos simples diferentes entre x e y.
  - Juntando esses dois caminhos, formaríamos um circuito.
  - ➜ Isso contradiz a definição de árvore.
  - ∴ Portanto, existe exatamente um caminho simples entre quaisquer dois vértices.
- Prova: (⇐) Se existe um único caminho simples entre cada par de vértices
  - Então o grafo é conexo, pois sempre há um caminho entre dois vértices.
  - Suponha que exista um circuito simples contendo x e y.
  - Nesse circuito haveria dois caminhos simples distintos entre x e y (seguindo o ciclo por lados diferentes).
  - ➜ Isso contradiz a hipótese de unicidade do caminho.
  - ∴ Portanto, o grafo não possui circuitos e é conexo.

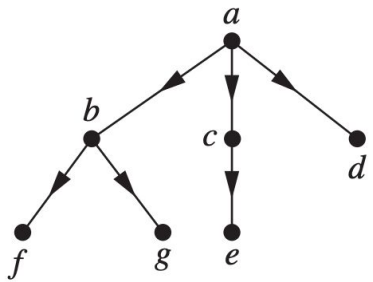
Árvore Enraizada

- <u>Definição:</u> Uma árvore enraizada é uma árvore na qual um vértice é designado como raiz, e todas as arestas são orientadas para **fora** da raiz.

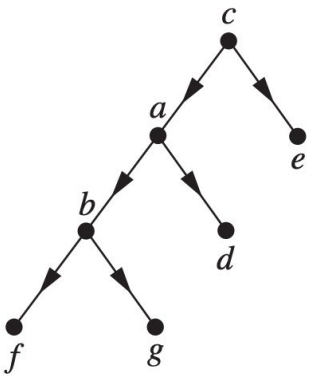
T



Com raiz *a*

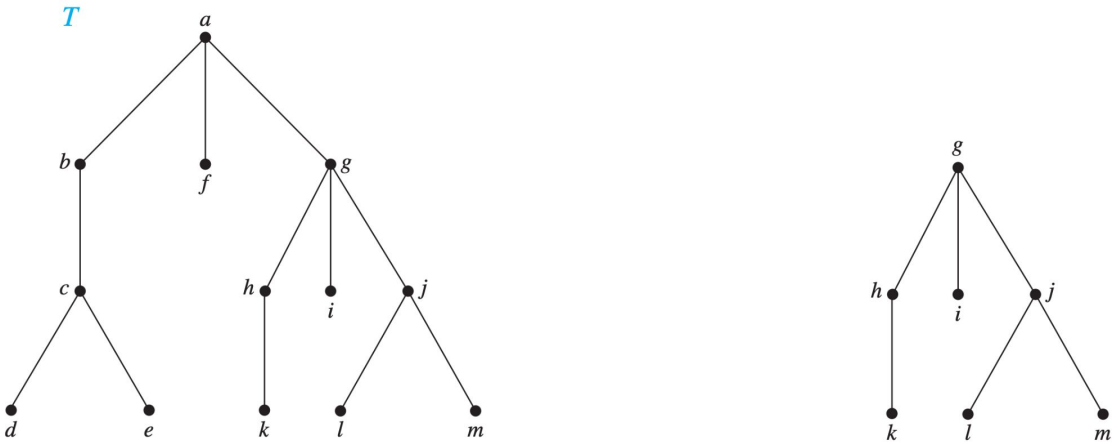


Com raiz *C*



Subárvore

- <u>Definição:</u> Se a é um vértice de uma árvore, a subárvore enraizada em a é o subgrafo formado por a, por todos os seus descendentes e pelas arestas que conectam esses vértices.



Árvore enraizadas ordenadas

- Definição: É uma árvore enraizada em que os filhos de cada vértice interno possuem uma ordem definida.
  - Nos desenhos, os filhos de cada vértice são apresentados da esquerda para a direita, seguindo essa ordem.
  - A forma convencional de desenhar árvores enraizadas já induz naturalmente uma ordenação das arestas.
- Exemplo: Árvore Binária (*Ordered Binary Tree / Binary Tree*)
  - É um tipo especial de árvore enraizada ordenada.
  - Cada vértice interno pode ter até dois filhos:
    - Filho esquerdo (left child)
      - A árvore com raiz no filho esquerdo é chamada de subárvore esquerda.
    - Filho direito (right child)
      - A árvore com raiz no filho direito é chamada de subárvore direita.

Propriedades das Árvores

- Teorema: Uma árvore com n vértices possui $n-1$ arestas.

Propriedades das Árvores (1)

- Prova (por indução matemática):
  - Passo base
    - Para n=1:
      - Uma árvore com **1 vértice** não possui arestas.
      - Logo, $0 = 1 - 1$.
      - ➜ O teorema vale para n=1.

Propriedades das Árvores (2)

- Prova (por indução matemática):
  - Passo indutivo
    - Hipótese de indução: Suponha que toda árvore com k vértices tem $k-1$ arestas.
    - Passo:
      - Considere uma árvore T com $\boldsymbol{k+1}$ **vértices**.
      - Como T é finita, existe uma **folha** $\boldsymbol{v}$.
      - Seja w o **pai de** $\boldsymbol{v}$.
      - Removendo v e a aresta (w,v) obtemos uma árvore T' com $\boldsymbol{k}$ **vértices**.
  - Pela hipótese de indução:
    - T' tem $k-1$ arestas
  - Adicionando novamente a aresta (w,v):
    - T tem k arestas
  - Conclui-se que uma árvore com $k+1$ vértices tem k arestas.

Propriedades das Árvores (3)

- Contagem de Vértices em Árvores m-árias Completas
- Definição: Em uma árvore m-ária completa *(full m-ary tree)*, podemos determinar o número total de vértices a partir do número de vértices internos.
- Teorema: Uma árvore m-ária completa com i vértices internos possui $n = mi + 1$ vértices no total.
  - m: número máximo de filhos que cada vértice interno pode ter
  - i: número de vértices internos
  - n: número total de vértices da árvore

Propriedades das Árvores (4)

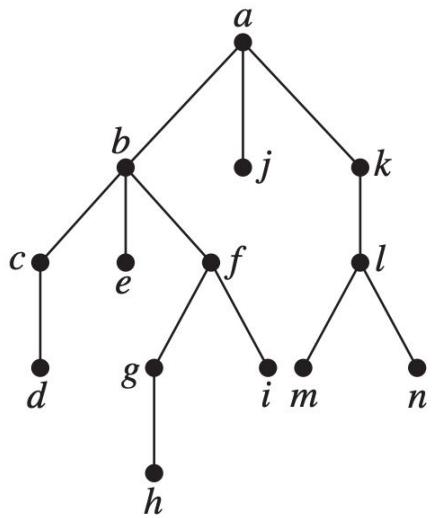
- Prova:
  - Considere uma árvore m-ária completa.
    - Todo vértice, exceto a raiz, é filho de algum vértice interno.
    - Se existem i vértices internos e cada um possui m filhos, então existem: mi vértices na árvore exceto a raiz.
    - Incluindo a raiz, o número total de vértices é: $n = mi + 1$
  - ➜ Portanto, uma árvore m-ária completa com i vértices internos possui $n = mi+1$ vértices.

Propriedades das Árvores (5): Árvores m-árias Balanceadas

- Em muitas aplicações, é desejável utilizar árvores enraizadas balanceadas, nas quais os caminhos nas subárvores possuem comprimentos aproximadamente iguais.
- <u>Definições:</u>
  - Nível (level) de um vértice v:
    - É o comprimento do caminho único da raiz até v.
  - Nível da raiz:
    - Definido como 0.
  - Altura (height) de uma árvore enraizada:
    - É o maior nível entre todos os vértices da árvore.
      - Equivalente: O comprimento do maior caminho da raiz até qualquer vértice da árvore.

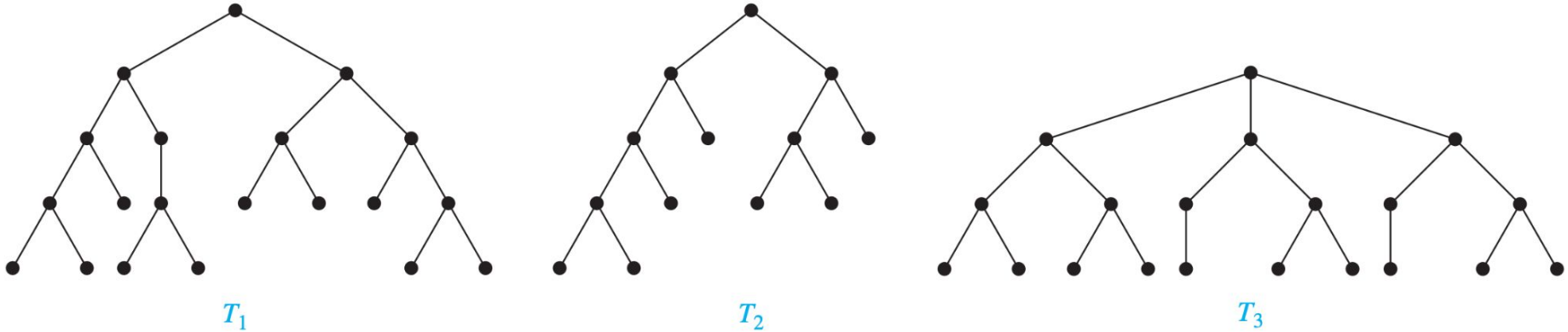
Propriedades das Árvores (6): Árvores m-árias Balanceadas (1)

- Determine o nível de cada vértice na árvore enraizada mostrada na figura. Qual é a altura dessa árvore?
  - Nível 0: a (raiz)
  - Nível 1: b, j, k
  - Nível 2: c, e, f, l
  - Nível 3: d, g, i, m, n
  - Nível 4: h
  - ➜ Como o maior nível entre os vértices é 4, a altura da árvore é 4.



Propriedades das Árvores (7): Árvores m-árias Balanceadas (2)

- Quais das árvores enraizadas mostradas na figura são balanceadas?
  - T_1 é balanceada, pois todas as suas folhas estão nos níveis 3 e 4.
  - T_2 não é balanceada, pois possui folhas nos níveis 2, 3 e 4.
  - T_3 é balanceada, pois todas as suas folhas estão no nível 3.



Propriedades das Árvores (7): Árvores m-árias Balanceadas (3)

- Definição (ou quase): Árvore balanceada é uma árvore em que as subárvores dos nós têm alturas aproximadamente iguais (comumente 1), garantindo menor altura total e operações eficientes.
  - Para cada nó v: $|\hbar(\text{esquerda}) - \hbar(\text{direita})| \leq 1$, em que $\hbar$ representa a altura da subárvore.
- Isso é importante porque uma árvore balanceada evita ficar "torta" e permite operações como busca, inserção e remoção com melhor desempenho, geralmente em tempo: $O(\log n)$
- Exemplo:
  - Balanceada: os nós ficam distribuídos de forma mais uniforme.
  - Desbalanceada: a árvore fica parecida com uma lista encadeada.

Propriedades das Árvores (8): # de folhas em uma árvore m-ária

- Teorema: em uma árvore m-ária de altura h, o número de folhas é **no máximo** m^k.
- Ideia da Prova (Indução na altura)
- Passo base (h = 1)
  - A árvore possui uma raiz com no máximo m filhos.
  - Todos os filhos são folhas. ⇒ número máximo de folhas = m = m^1
- Hipótese de indução
  - Suponha que toda árvore m-ária de altura menor que h possui no máximo m^{h-1} folhas.
- Passo indutivo
  - Considere uma árvore m-ária T de altura h.
  - Removendo as arestas da raiz para seus filhos, obtemos até m subárvores.
  - Cada sub árvore tem altura no máximo $h-1$.
- Pela hipótese de indução: cada subárvore tem no máximo m^{h-1} folhas
- Como existem no máximo m subárvores: $m \cdot m^{h-1}=m^h$
- ➜ Portanto, uma árvore m-ária de altura h possui no máximo m^h folhas.

Propriedades das Árvores (9): altura da árvore m-ária

- Colorário:
  - Se uma árvore m-ária de altura h possui l folhas, então $h \geq \lceil log_m l \rceil$
  - Se a árvore m-ária for completa (full) e balanceada, então $h = \lceil \log_m l \rceil$
    - onde $\lceil x \rceil$ representa o teto (ceiling) de x, isto é, o menor inteiro maior ou igual a x.
- Ideia da prova:
  - Pelo teorema anterior, sabemos que: $l \leq m^k$
    - Aplicando logaritmo na base m: $\log_m l \leq h$
    - Como h é inteiro: $h \geq \lceil \log_m l \rceil$
  - Caso especial (árvore completa e balanceada):
    - As folhas estão apenas nos níveis h ou $h-1$.
    - Assim: $m^{h-1} < l \leq m^h$
    - Tomando logaritmos: $h-1 < \log_m l \leq h$
    - Logo, $h = \lceil log_m l \rceil$

Aplicações

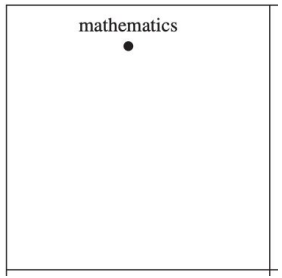
- Árvores são estruturas fundamentais para resolver diversos problemas computacionais.
- Aqui consideramos três tipos de problemas que podem ser estudados usando árvores:
  - Armazenamento e busca em listas
    - Como armazenar itens em uma lista de forma que sejam facilmente localizados?
  - Tomada de decisões
    - Qual sequência de decisões deve ser feita para encontrar um objeto com uma determinada propriedade em uma coleção?
  - Codificação eficiente de caracteres
    - Como representar um conjunto de caracteres usando sequências de bits de forma eficiente?

Aplicações: Árvores Binárias de Busca (*Binary Search Trees*)

- A busca de itens em uma lista é uma das tarefas mais importantes na ciência da computação.
  - Nosso objetivo é implementar um algoritmo eficiente de busca quando os itens estão ordenados.
- Uma forma de fazer isso é usando uma *Binary Search Trees*.
- <u>Definição:</u>
  - Cada vértice tem no máximo dois filhos:
    - filho esquerdo
    - filho direito
  - Cada vértice possui uma chave (key) associada.
  - As chaves obedecem à seguinte propriedade:
    - *chaves da subárvore esquerda < chave do vértice < chaves da subárvore direita*

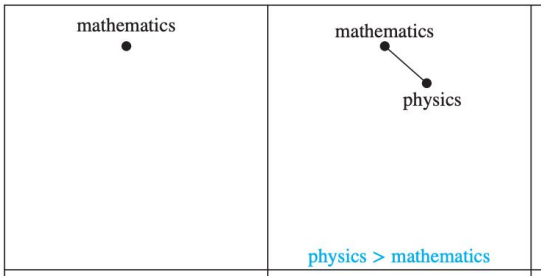
Aplicações: Árvores Binárias de Busca (*Binary Search Trees*) (1)

- Exemplo:
  - Construa uma árvore binária de busca para as palavras: *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, *chemistry* (utilizando ordem alfabética).



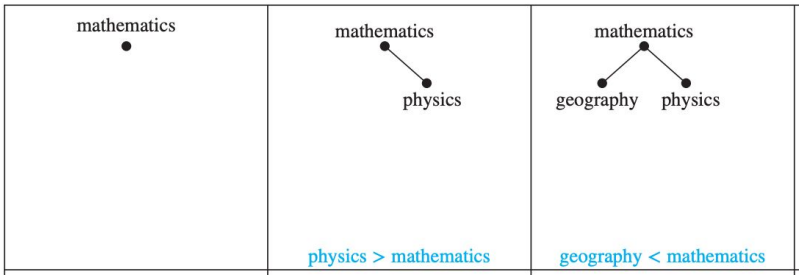
Aplicações: Árvores Binárias de Busca (*Binary Search Trees*) (2)

- Exemplo:
  - Construa uma árvore binária de busca para as palavras: *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, *chemistry* (utilizando ordem alfabética).



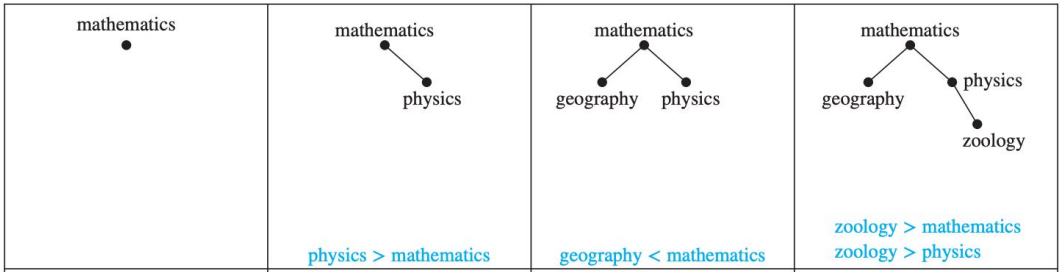
Aplicações: Árvores Binárias de Busca (*Binary Search Trees*) (3)

- Exemplo:
  - Construa uma árvore binária de busca para as palavras: *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, *chemistry* (utilizando ordem alfabética).



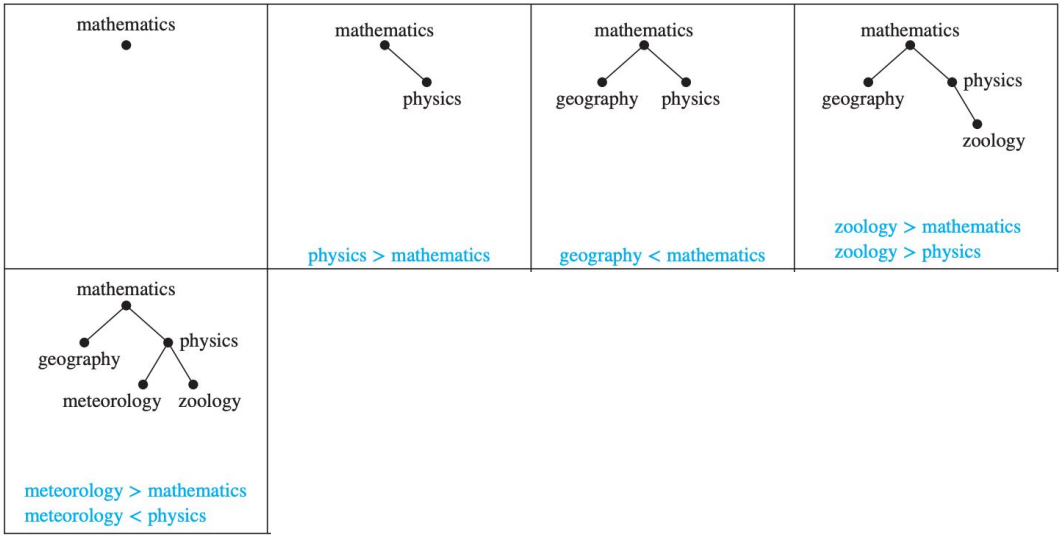
Aplicações: Árvores Binárias de Busca (*Binary Search Trees*) (4)

- Exemplo:
  - Construa uma árvore binária de busca para as palavras: *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, *chemistry* (utilizando ordem alfabética).



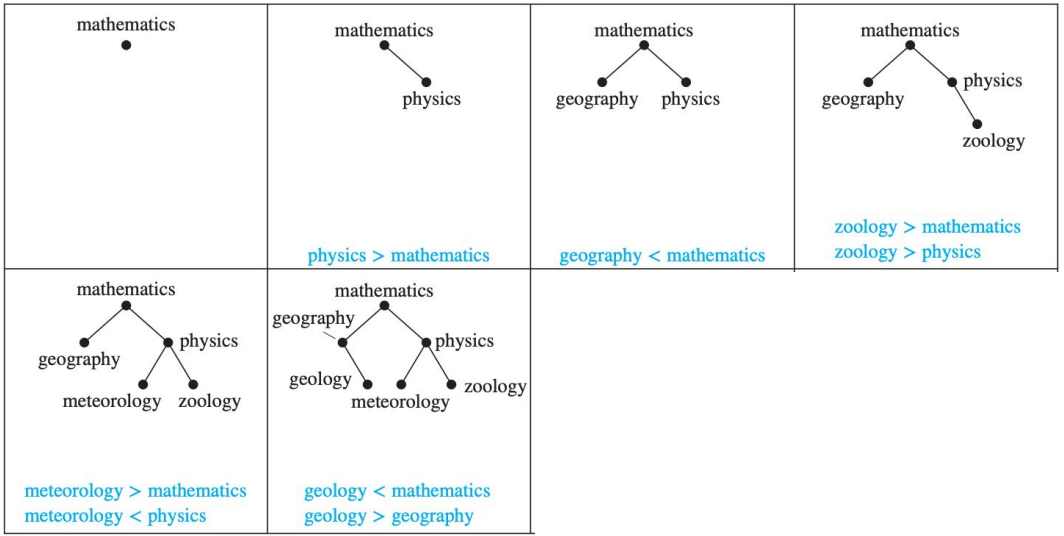
Aplicações: Árvores Binárias de Busca (*Binary Search Trees*) (5)

- Exemplo:
  - Construa uma árvore binária de busca para as palavras: *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, *chemistry* (utilizando ordem alfabética).



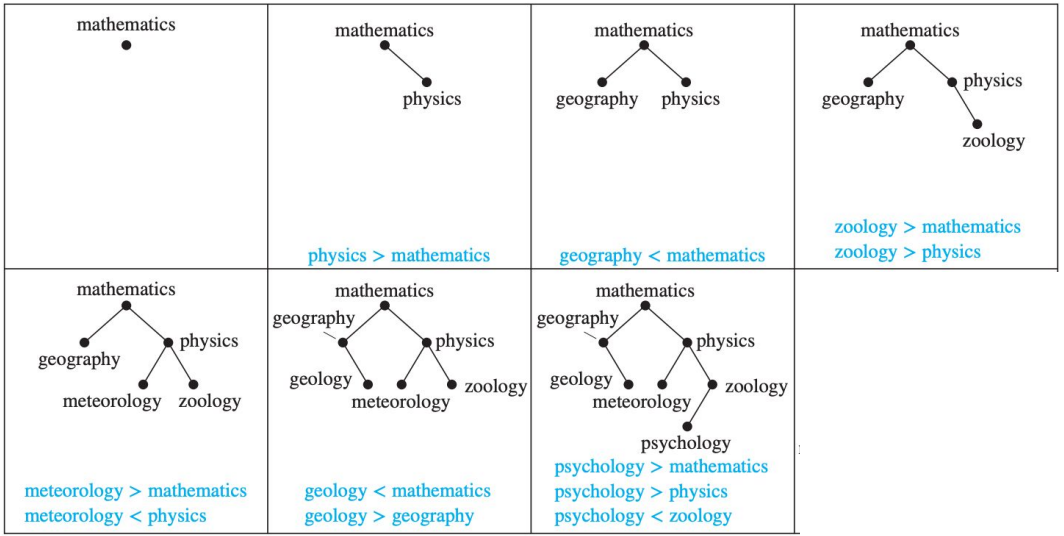
Aplicações: Árvores Binárias de Busca (*Binary Search Trees*) (6)

- Exemplo:
  - Construa uma árvore binária de busca para as palavras: *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, *chemistry* (utilizando ordem alfabética).



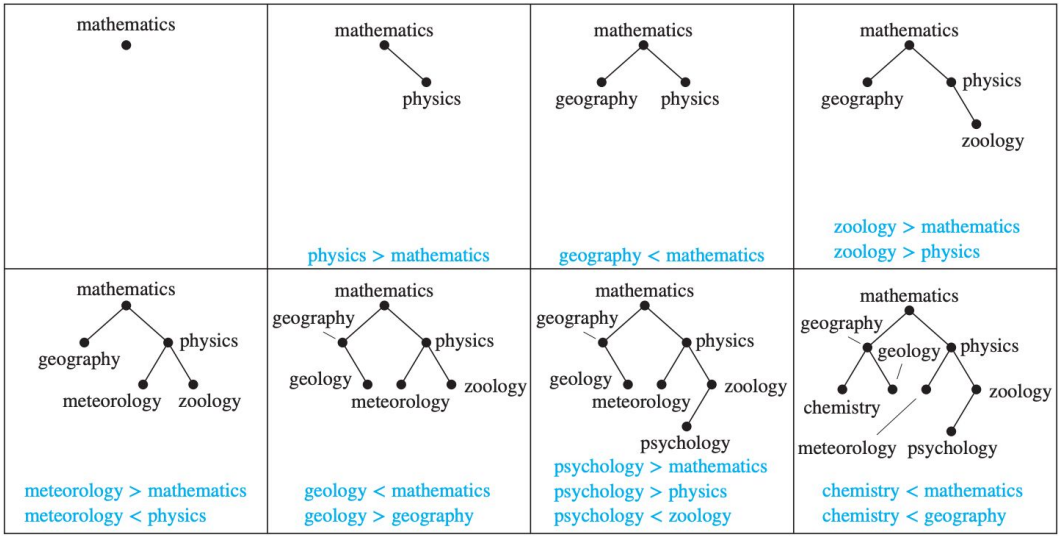
Aplicações: Árvores Binárias de Busca (*Binary Search Trees*) (7)

- Exemplo:
  - Construa uma árvore binária de busca para as palavras: *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, *chemistry* (utilizando ordem alfabética).



Aplicações: Árvores Binárias de Busca (*Binary Search Trees*) (8)

- Exemplo:
  - Construa uma árvore binária de busca para as palavras: *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, *chemistry* (utilizando ordem alfabética).



Aplicações: BST Algoritmo inserir/buscar

ALGORITHM 1 Locating an Item in or Adding an Item to a Binary Search Tree.

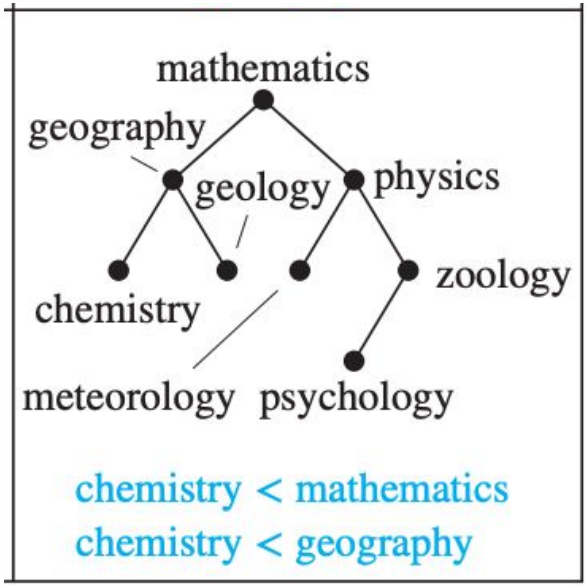
procedure *insertion*(T: binary search tree,
x: item)
v := root of T
{a vertex not present in T has the value *null* }
while $v \neq$ *null* and *label*(v) $\neq x$
  if $x <$ *label*(v) **then**
    if left child of $v \neq$ *null* **then** v := left child of v
    else add *new vertex* as a left child of v and set v := *null*
  else
    if right child of $v \neq$ *null* **then** v := right child of v
    else add *new vertex* as a right child of v and set v := *null*
if root of T = *null* **then** add a vertex v to the tree and label it with x
else if v is null or *label*(v) $\neq x$ **then** label *new vertex* with x and let v be this new vertex
return v {v = location of x}

Aplicações: BST Algoritmo inserir/buscar (1)

ALGORITHM 1 Locating an Item in or Adding an Item to a Binary Search Tree.

Inserir: oceanography

procedure *insertion*(T: binary search tree,
x: item)
v := root of T
{a vertex not present in T has the value *null* }
while $v \neq$ *null* and *label*(v) $\neq x$
  if $x <$ *label*(v) **then**
    if left child of $v \neq$ *null* **then** v := left child of v
    else add *new vertex* as a left child of v and set v := *null*
  else
    if right child of $v \neq$ *null* **then** v := right child of v
    else add *new vertex* as a right child of v and set v := *null*
if root of T = *null* **then** add a vertex v to the tree and label it with x
else if v is null or *label*(v) $\neq x$ **then** label *new vertex* with x and let v be this new vertex
return v {v = location of x}

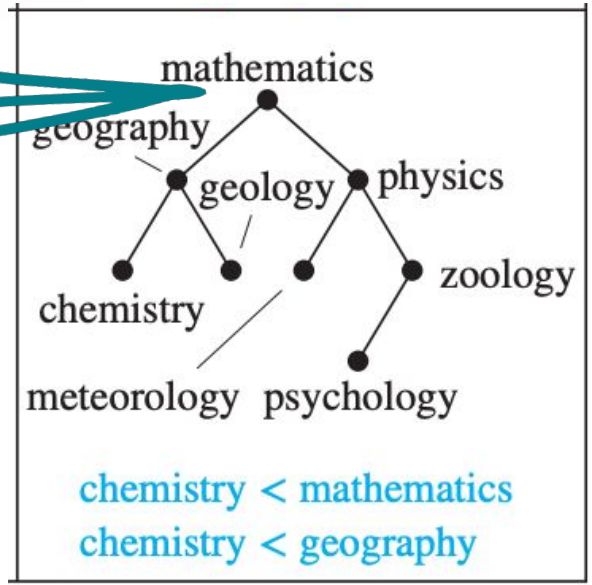


Aplicações: BST Algoritmo inserir/buscar (2)

ALGORITHM 1 Locating an Item in or Adding an Item to a Binary Search Tree.

```
procedure insertion(T: binary search tree,
x: item)
v := root of T
{a vertex not present in T has the value null }
while v ≠ null and label(v) ≠ x
    if x < label(v) then
        if left child of v ≠ null then v := left child of v
        else add new vertex as a left child of v and set v := null
    else
        if right child of v ≠ null then v := right child of v
        else add new vertex as a right child of v and set v := null
if root of T = null then add a vertex v to the tree and label it with x
else if v is null or label(v) ≠ x then label new vertex with x and let v be this new vertex
return v {v = location of x}
```

Inserir: oceanography

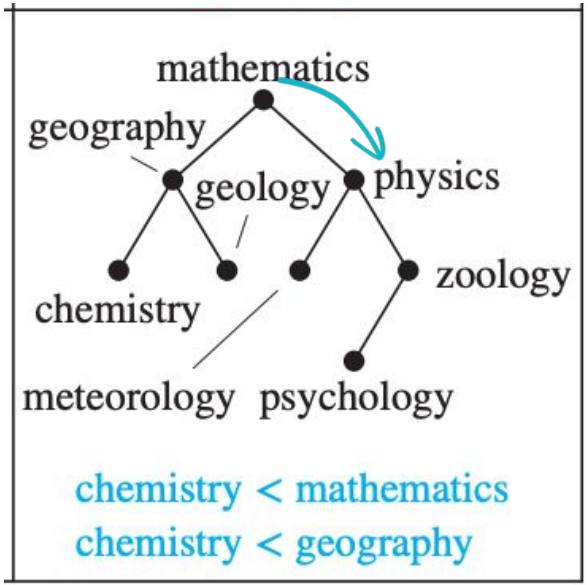


Aplicações: BST Algoritmo inserir/buscar (3)

ALGORITHM 1 Locating an Item in or Adding an Item to a Binary Search Tree.

Inserir: oceanography

procedure *insertion*(T: binary search tree,
x: item)
v := root of T
{a vertex not present in T has the value *null* }
while $v \neq$ *null* and *label*(v) $\neq x$
  if $x <$ *label*(v) **then**
    if left child of $v \neq$ *null* **then** v := left child of v
    else add *new vertex* as a left child of v and set v := *null*
  else
    if right child of $v \neq$ *null* **then** v := right child of v
    else add *new vertex* as a right child of v and set v := *null*
if root of T = *null* **then** add a vertex v to the tree and label it with x
else if v is null or *label*(v) $\neq x$ **then** label *new vertex* with x and let v be this new vertex
return v {v = location of x}

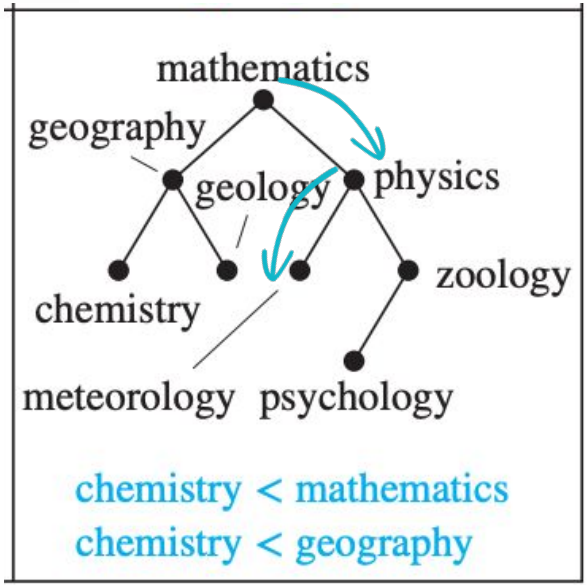


Aplicações: BST Algoritmo inserir/buscar (4)

ALGORITHM 1 Locating an Item in or Adding an Item to a Binary Search Tree.

Inserir: oceanography

```
procedure insertion(T: binary search tree,
x: item)
v := root of T
{a vertex not present in T has the value null }
while v ≠ null and label(v) ≠ x
    if x < label(v) then
        if left child of v ≠ null then v := left child of v
        else add new vertex as a left child of v and set v := null
    else
        if right child of v ≠ null then v := right child of v
        else add new vertex as a right child of v and set v := null
if root of T = null then add a vertex v to the tree and label it with x
else if v is null or label(v) ≠ x then label new vertex with x and let v be this new vertex
return v {v = location of x}
```

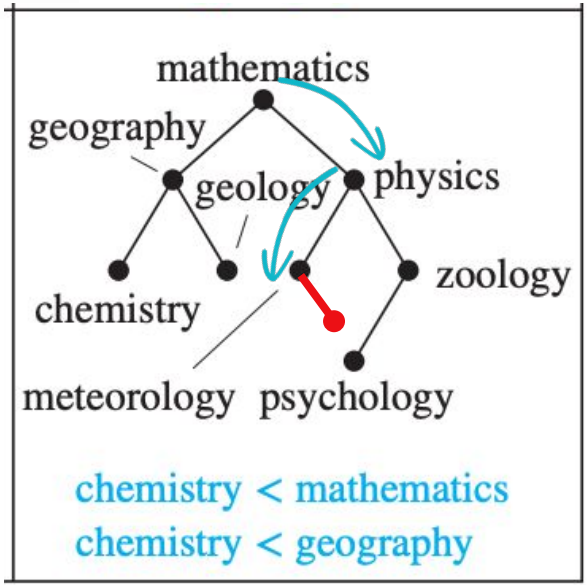


Aplicações: BST Algoritmo inserir/buscar (5)

ALGORITHM 1 Locating an Item in or Adding an Item to a Binary Search Tree.

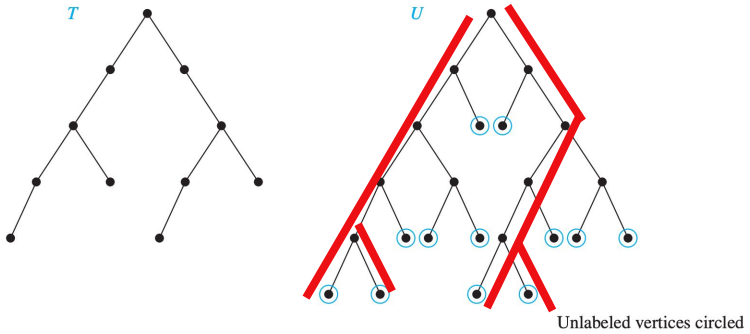
Inserir: oceanography

```
procedure insertion(T: binary search tree,
x: item)
v := root of T
{a vertex not present in T has the value null }
while v ≠ null and label(v) ≠ x
    if x < label(v) then
        if left child of v ≠ null then v := left child of v
        else add new vertex as a left child of v and set v := null
    else
        if right child of v ≠ null then v := right child of v
        else add new vertex as a right child of v and set v := null
if root of T = null then add a vertex v to the tree and label it with x
else if v is null or label(v) ≠ x then label new vertex with x and let v be this new vertex
return v {v = location of x}
```



Aplicações: BST Algoritmo inserir/buscar - Complexidade

- Considere uma árvore binária de busca T com n itens. Podemos construir uma árvore binária completa U adicionando vértices não rotulados sempre que necessário, de modo que todo vértice interno tenha dois filhos.
- Ideia da Análise
  - O maior número de comparações para inserir um item corresponde ao comprimento do maior caminho da raiz até uma folha.
  - Os vértices internos de U são exatamente os vértices de T.
  - Logo, U possui: n vértices internos e n + 1 folhas

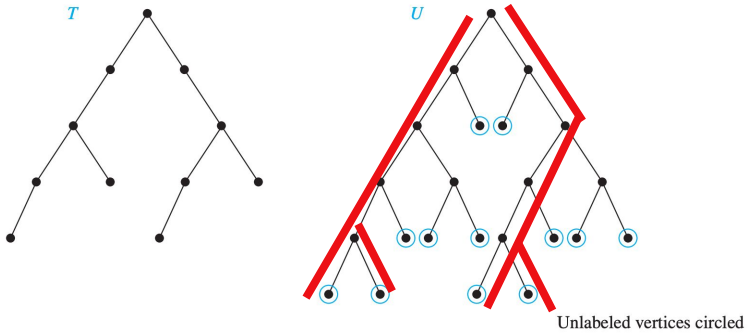


Aplicações: BST Algoritmo inserir/buscar - Complexidade

- Altura da Árvore
  - Usando resultados anteriores: $h \geq \lceil \log_2 (n + 1) \rceil$
  - Portanto, existe um item cuja inserção requer pelo menos $\lceil \log_2 (n + 1) \rceil$ comparações.
- Caso Balanceado
  - Se a árvore binária de busca é balanceada, então: $h = \lceil \log_2 (n + 1) \rceil$

➜ Assim, buscar ou inserir um item requer no máximo $O(\log n)$ comparações.

➜ Árvores binárias de busca podem se tornar **desbalanceadas conforme novos itens são inseridos**. Por isso, existem algoritmos que **rebalanceiam a árvore automaticamente (ex.: AVL, Red-Black Trees)**.



Aplicações: Árvores de Decisão (*Decision Trees*)

- Árvores enraizadas podem ser usadas para modelar problemas em que uma sequência de decisões leva a uma solução.
- Ideia
  - Cada vértice interno representa uma decisão.
  - Cada aresta representa um possível resultado da decisão.
  - Cada folha representa uma solução possível.
- Exemplo de Aplicação
  - Em uma árvore binária de busca, cada comparação decide:
    - ir para a subárvore esquerda, ou
    - ir para a subárvore direita,
  - Até localizar o item desejado.

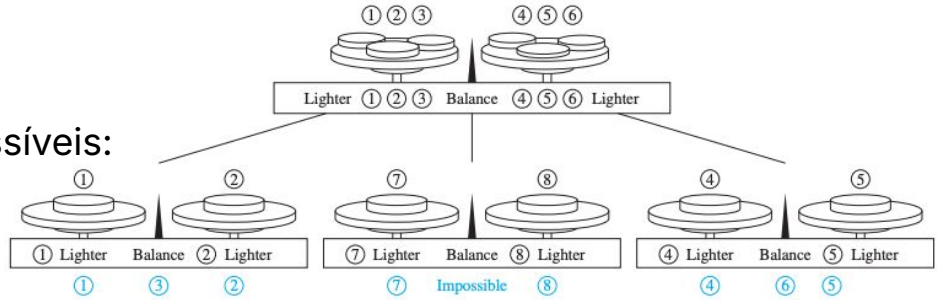
➜ Assim, em uma árvore de decisão, os caminhos da raiz até as folhas representam as possíveis sequências de decisões que levam às soluções do problema.

Aplicações: Árvores de Decisão (*Decision Trees*) (1)

- Exemplo: Encontrando a Moeda Falsa com Árvore de Decisão
- Problema: Há 8 moedas: 7 têm o mesmo peso e 1 é falsa, sendo mais leve que as demais. Quantas pesagens em uma balança de dois pratos são necessárias para identificar a moeda falsa? E qual algoritmo pode ser usado?

Aplicações: Árvores de Decisão (*Decision Trees*) (2)

- Ideia da Solução
  - Em cada pesagem, há 3 resultados possíveis:
    - prato esquerdo mais leve
    - Equilíbrio
    - prato direito mais leve



  - Logo, o processo pode ser modelado por uma árvore de decisão ternária.
  - Como existem 8 possibilidades (qualquer uma das 8 moedas pode ser a falsa), a árvore precisa ter pelo menos 8 folhas.
  - Assim, o número mínimo de pesagens é a altura mínima h tal que: $3^h \geq 8$
    - ou seja, $h \geq \lceil \log_3 8 \rceil = 2$

➜ Portanto, são necessárias pelo menos 2 pesagens.

* Árvores de Decisão e Complexidade de Algoritmos de Ordenação Baseados em Comparação

- Existem muitos algoritmos de ordenação.
- Para avaliar se um algoritmo é eficiente, analizamos sua **complexidade**.
- Uma forma de estudar essa complexidade é usando árvores de decisão.
- Ideia Principal
  - Algoritmos de ordenação frequentemente usam comparações entre dois elementos.
  - Cada comparação reduz o conjunto de ordenações possíveis.
  - Se temos n elementos, existem: $n!$ possíveis ordenações (permutações - uma delas é a ordenada!).

* Árvores de Decisão e Complexidade de Algoritmos de Ordenação Baseados em Comparação (1)

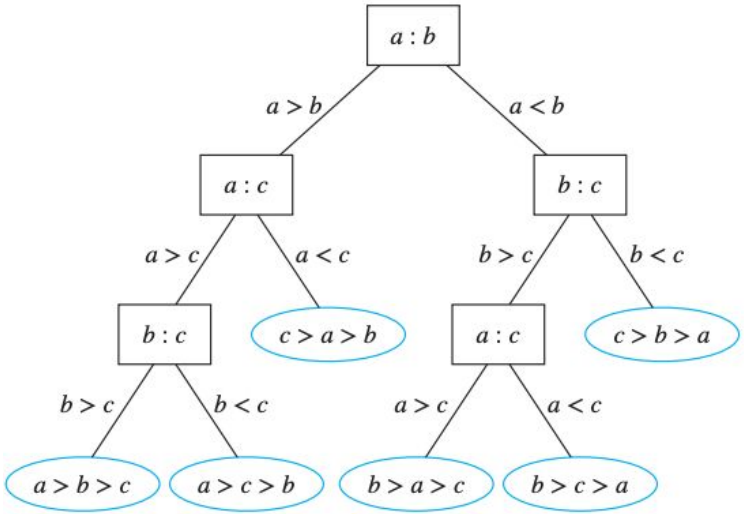
- Modelo com Árvores de Decisão
  - Um algoritmo de ordenação baseado em comparações pode ser representado por uma árvore binária de decisão:
    - Vértices internos → comparações entre dois elementos
    - Arestas → resultados possíveis da comparação
    - Folhas → uma das $n!$ possíveis ordenações

➜ Consequência: A análise dessa árvore permite obter um limite inferior para o número de comparações necessárias no pior caso para ordenar n elementos.

* Árvores de Decisão e Complexidade de Algoritmos de Ordenação Baseados em Comparação (2)

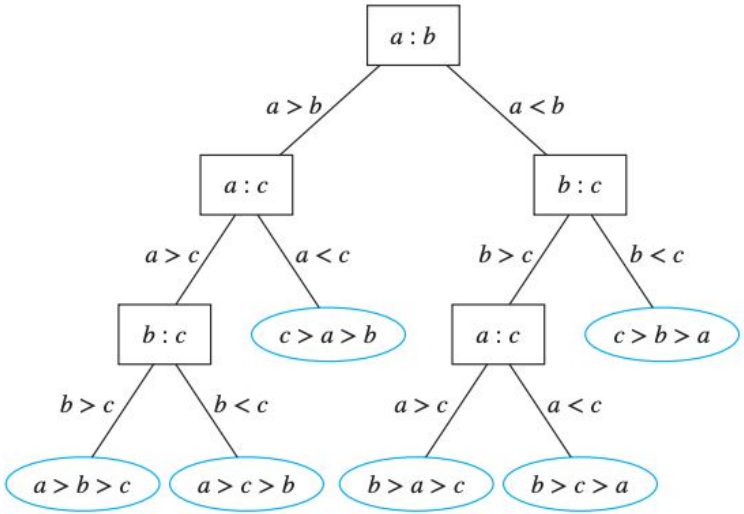
- Exemplo: Árvore de Decisão para Ordenar Três Elementos
  - Considere três elementos distintos: a, b, c.
  - Podemos representar um algoritmo de ordenação baseado em comparações usando uma árvore de decisão.
- Estrutura da Árvore
  - Vértices internos: representam comparações entre dois elementos
    - exemplo: comparar: a e b
  - Arestas: representam o resultado da comparação
    - $a < b$ ou $a > b$
  - Folhas: representam a ordenação final dos elementos
- Como existem 3 elementos, existem: 3! = 6 possíveis ordenações.

➜ Logo, a árvore de decisão precisa ter 6 folhas.



* Árvores de Decisão e Complexidade de Algoritmos de Ordenação Baseados em Comparação (3)

- Complexidade:
  - O número de comparações necessárias no pior caso corresponde ao:
    - comprimento do maior caminho da raiz até uma folha
    - ou seja, à altura da árvore de decisão.
  - Como uma árvore binária com $n!$ folhas tem altura pelo menos: $\lceil \log_2(n!) \rceil$
  - Segue que qualquer algoritmo de ordenação baseado em comparações precisa de pelo menos $\lceil \log_2(n!) \rceil$ comparações no pior caso.



* Árvores de Decisão e Complexidade de Algoritmos de Ordenação Baseados em Comparação (4)

- Teorema: Um algoritmo de ordenação baseado em comparações binárias requer pelo menos $\lceil \log_2(n!) \rceil$ comparações no pior caso.
  - Interpretação:
    - Cada algoritmo de ordenação baseado em comparações pode ser representado por uma árvore de decisão binária.
    - As folhas dessa árvore correspondem às possíveis permutações dos n elementos.
    - Como existem $n!$ permutações, a árvore precisa ter $n!$ folhas.
  - Logo, a altura da árvore (número máximo de comparações) é pelo menos: $\lceil \log_2(n!) \rceil$
  - ➜ Consequência:
    - Sabemos que: $\log(n!) = \Theta(n \log n)$
    - Portanto, qualquer algoritmo de ordenação baseado em comparações tem limite inferior: $\Omega(n \log n)$ no pior caso. Prova (link)

* Árvores de Decisão e Complexidade de Algoritmos de Ordenação Baseados em Comparação (5)

- Colorário: O número de comparações usadas por um algoritmo de ordenação baseado em comparações binárias para ordenar n elementos é $\Omega(n \log n)$
  - Ideia:
    - Pelo Teorema 1, um algoritmo de ordenação baseado em comparações requer pelo menos $\lceil \log_2(n!) \rceil$ comparações.
    - Sabemos que $\log(n!) = \Theta(n \log n)$

➜ Portanto, qualquer algoritmo de ordenação baseado em comparações possui limite inferior assintótico $\Omega(n \log n)$ para o número de comparações necessárias.

* Árvores de Decisão e Complexidade de Algoritmos de Ordenação Baseados em Comparação (6)

- Consequência do Corolário anterior: Se um algoritmo de ordenação baseado em comparações usa $\Theta(n \log n)$ comparações no pior caso, então ele é ótimo, pois nenhum outro algoritmo desse tipo pode ter melhor complexidade no pior caso.
  - Exemplo: Merge Sort é ótimo nesse sentido.
- Complexidade no Caso Médio:
  - Também podemos analisar o número médio de comparações.
    - Esse valor corresponde à profundidade média das folhas na árvore de decisão do algoritmo.
    - Para uma árvore binária com N vértices, a profundidade média das folhas é: $\Omega(\log N)$
    - Como no problema de ordenação $N = n!$
      - $\Omega(\log(n!)) = \Omega(n \log n)$
- Teorema: O número médio de comparações usado por um algoritmo de ordenação baseado em comparações para ordenar n elementos é $\Omega(n \log n)$

➜ Mesmo no caso médio, algoritmos de ordenação baseados em comparações não podem ser melhores que $n \log n$ em termos assintóticos.

Aplicações: Códigos de Prefixo (*Prefix Codes*)

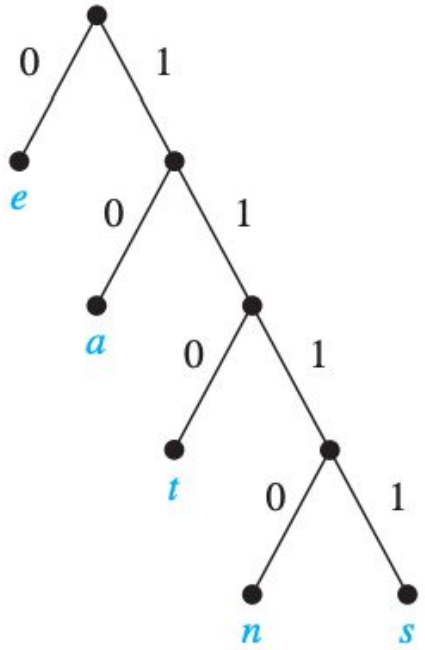
- Motivação
  - Queremos codificar letras usando sequências de bits.
    - 26 letras → podem ser codificadas com 5 bits (pois $2^5 = 32$)
    - **Problema: isso pode não ser eficiente**
- Ideia: usar códigos de comprimentos diferentes
  - Letras frequentes → códigos curtos
  - Letras raras → códigos longos

Aplicações: Códigos de Prefixo (*Prefix Codes*) (1)

- Problema
  - Ao usar códigos de tamanhos diferentes, pode surgir ambiguidade:
  - Exemplo:
    - e → 0, a → 1, t → 01
    - A sequência 0101 pode representar:
      - eat, tea, eaea, etc.
- Solução: Códigos de Prefixo
  - Um código de prefixo é um código em que:
    - Nenhum código é prefixo de outro.
    - ➜ Isso garante decodificação única.
- Exemplo de é um código de prefixo.
  - e → 0
  - a → 10
  - t → 11
- Exemplo de decodificação:
  - 10110 → ate

Aplicações: Códigos de Prefixo (*Prefix Codes*) (2)

- Representação com Árvores
  - Um código de prefixo pode ser representado por uma árvore binária:
    - Aresta esquerda → 0
    - Aresta direita → 1
    - Folhas → caracteres
- O código de um símbolo é o caminho da raiz até a folha.
- Decodificação
  1. Comece na raiz
  2. Leia os bits:
     - 0 → esquerda
     - 1 → direita
  3. Ao chegar em uma folha, identifique o símbolo
  4. Volte à raiz e continue



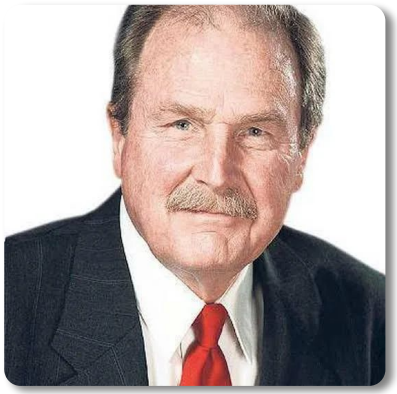
* Codificação de Huffman

DAVID A. HUFFMAN (1925–1999) David Huffman cresceu em Ohio. Aos 18 anos, recebeu seu diploma de Bacharel em Engenharia Elétrica pela Ohio State University. Em seguida, serviu na Marinha dos Estados Unidos como oficial de manutenção de radar em um destróier cuja missão era limpar minas em águas asiáticas após a Segunda Guerra Mundial. Mais tarde, obteve seu mestrado pela Ohio State e seu doutorado em engenharia elétrica pelo MIT. Huffman ingressou no corpo docente do MIT em 1953, onde permaneceu até 1967, quando se tornou membro fundador do departamento de ciência da computação da Universidade da Califórnia em Santa Cruz. Ele desempenhou um papel importante no desenvolvimento desse departamento e passou o restante de sua carreira lá, aposentando-se em 1994.

Huffman é reconhecido por suas contribuições à teoria da informação e codificação, ao projeto de sinais para radar e comunicações, e ao desenvolvimento de procedimentos para circuitos lógicos assíncronos. Seu trabalho com superfícies de curvatura zero o levou a desenvolver técnicas originais para dobrar papel e vinil em formas incomuns, consideradas obras de arte por muitos e exibidas publicamente em várias exposições. No entanto, Huffman é mais conhecido pelo desenvolvimento do que hoje é chamado de codificação de Huffman, resultado de um trabalho que escreveu durante seu período de pós-graduação no MIT.

Huffman gostava de explorar a natureza, fazer trilhas e viajar extensivamente. Ele se certificou como mergulhador autônomo já na faixa dos 60 anos. Também mantinha cobras venenosas como animais de estimação.

- Criador do algoritmo de Huffman (1951, MIT)
- Contribuições em:
  - Teoria da informação
  - Codificação
  - Compressão de dados
- Seu algoritmo é amplamente utilizado até hoje



* Codificação de Huffman: Algoritmo

- Dado um conjunto de símbolos com suas frequências, construir um código de prefixo ótimo, ou seja, que usa o menor número médio de bits.

* Codificação de Huffman: Algoritmo (1)

- Ideia do Algoritmo
  1. Comece com uma floresta de árvores, cada uma contendo:
     - um único símbolo
     - peso = frequência do símbolo
  2. Enquanto houver mais de uma árvore:
     - Escolha as duas árvores de menor peso
     - Combine-as em uma nova árvore:
       - novo nó raiz
       - peso = soma dos pesos
       - esquerda (0) e direita (1)
  3. Quando restar apenas uma árvore → árvore de Huffman
- Construção do Código
  - Cada símbolo está em uma folha
  - O código é o caminho da raiz até a folha
    - esquerda → 0
    - direita → 1

- Veja (link)

ALGORITHM 2 Huffman Coding.

procedure *Huffman*(C: symbols a_i with frequencies w_i, $i = 1, \ldots, n$)
F := forest of n rooted trees, each consisting of the single vertex a_i and assigned weight w_i
while F is not a tree
  Replace the rooted trees T and T' of least weights from F with $w(T) \geq w(T')$ with a tree having a new root that has T as its left subtree and T' as its right subtree. Label the new edge to T with 0 and the new edge to T' with 1.
  Assign $w(T) + w(T')$ as the weight of the new tree.
{the Huffman coding for the symbol a_i is the concatenation of the labels of the edges in the unique path from the root to the vertex a_i}

* Codificação de Huffman: Algoritmo (2)

- Exemplo
- Frequências:
  - A: 0.08, B: 0.10, C: 0.12, D: 0.15, E: 0.20, F: 0.35
- Codificação obtida:
  - A → 111
  - B → 110
  - C → 011
  - D → 010
  - E → 10
  - F → 00
- Número Médio de Bits
  - 3(0.08)+3(0.10)+3(0.12)+3(0.15)+2(0.20)+2(0.35)= 2.45

➜ Em média, 2.45 bits por símbolo

- Propriedades
  - Algoritmo guloso (greedy)
  - Produz um código ótimo
  - Nenhum código de prefixo usa menos bits (em média)
- Observações
  - Pode ser estendido para:
    - blocos de símbolos
    - códigos adaptativos (frequências desconhecidas)
- Amplamente usado em:
  - JPEG
  - compressão de arquivos